

BobaLink Browser Extension

Technical Specification

Version: 1.0.0

Date: 2026-06-06

Classification: Internal — Engineering & Architecture

Table of Contents

1. Executive Summary
2. System Overview
3. Functional Requirements
4. Non-Functional Requirements
5. System Architecture
6. Data Models
7. API Specifications
8. Security Architecture
9. Error Handling Strategy
10. Performance Considerations
11. Technology Stack
12. Deployment Architecture
13. Assumptions and Dependencies
14. Risks and Mitigations

1. Executive Summary

BobaLink is a cross-browser WebExtension (Manifest V3) designed to bridge discovery on the public internet and private download management infrastructure. It discovers magnet links and `.torrent` file references on any web page, deduplicates them, and transmits them to a Boba orchestration server or directly to a qBitTorrent client for management.

The extension addresses three core operational challenges:

- Discovery-to-Download Latency:** Users currently copy magnet links manually and paste them into download clients. BobaLink reduces this workflow by automating the process.
- Batch Operation Efficiency:** Tab groups containing multiple torrent page units, enabling bulk ingestion without per-tab interaction.
- Operational Resilience:** An offline-aware queue with exponential backoff ensures that network failures or service maintenance do not result in lost torrents.

BobaLink is built atop the WXT framework in TypeScript, targets Chrome (≥ 110), Yandex Browser, and adheres to WCAG 2.1 AA accessibility standards. All sensitive data is encrypted at rest using AES-256-GCM via the Web Crypto API.

Key Metrics at a Glance

Metric	Target
Extension Bundle Size (compressed)	≤ 350 KB

2. System Overview

BobaLink operates as a thin client within the browser's extension host environment, managing BitTorrent protocol operations itself; rather, it acts as an intelligent scraper and interface between the web browsing context and downstream download services.

2.1 High-Level Architecture

[Diagram: System Architecture Overview]

2.2 Core Workflows

Workflow 1: Single Magnet Link Detection and Forward

1. Content script observes the DOM mutation events on the active tab.
2. A magnet URI is matched against the regex pattern `^magnet:\?xt=urn:b`
3. The link is parsed into a structured `TorrentInfo` object.
4. The torrent is sent to the service worker via `chrome.runtime.sendMessage`
5. The service worker enqueues the item or sends it immediately based on configuration.
6. The target API (Boba or qBitTorrent) receives the torrent and initiates download.

Workflow 2: Tab Group Batch Ingestion

1. User triggers "Send Tab Group to Boba" from the popup or context menu.
2. Service worker enumerates all tabs in the specified tab group.

3. Functional Requirements

3.1 Torrent Detection and Parsing

FR-001: Detect Magnet Links on Any Web Page

Attribute	Value
Priority	P0 — Critical
Actor	Content Script
Trigger	DOM mutation (<code>MutationObserver</code>) or page load

The extension SHALL detect all magnet URI links (`href` attributes beginning with `magnet:`) in the DOM of any web page. Detection MUST occur dynamically as pages mutate (SPA) navigation.

Acceptance Criteria:

- Magnet links in dynamically injected content (e.g., React/Vue re-renders) are detected.
- Detection works across all `http:` and `https:` origins.
- Magnet links inside iframes are detected when `all_frames` is enabled.

FR-002: Detect .torrent File Links

Attribute	Value
-----------	-------

FR-003: Parse Magnet URI Parameters

Attribute	Value
Priority	P0 — Critical
Actor	Parser Module

The extension SHALL parse the following magnet URI parameters per BEP-0

Parameter	Name	Description
xt	Exact Topic	urn:btih: followed by 40-char l
dn	Display Name	Human-readable torrent name
tr	Tracker URL	Announce URL(s); may be repeated
xl	Exact Length	Total size in bytes (optional)
ws	Web Seed	Web seed URL(s) (optional)
x.pe	Peer Address	Known peer host:port (optiona

Validation Rules:

- xt MUST be present and contain a valid 40-character hexadecimal infohash
- dn SHOULD be present; if absent, fallback to "Unknown Torrent" .
- tr entries MUST be valid HTTP(S) URLs; malformed trackers are logged

The extension SHALL download `.torrent` files via `fetch()` with the following constraints:

- Maximum file size: 10 MB (configurable via `maxTorrentFileSize` option)
- Timeout: 30 seconds.
- CORS policy: Falls back to proxying via Boba server when direct fetch is blocked

The downloaded file SHALL be parsed using a Bencode decoder to extract:

- `info.name` — torrent name
- `info.piece length` — piece size
- `info.length` (or `info.files` for multi-file) — total size
- `announce` and `announce-list` — tracker URLs

FR-005: Compute Infohash from Torrent Data

Attribute	Value
Priority	P1 — High
Actor	Crypto Module

For `.torrent` files, the extension SHALL compute the SHA-1 hash over the bencode dictionary to produce the 40-character hex infohash. This MUST match the `infohash` field, if available.

```
// Pseudocode for infohash computation
const infoBencoded = bencode.encode(torrent.info);
const infohashBuffer = await crypto.subtle.digest('SHA-1', infoBencoded);
const infohash = hexEncode(infohashBuffer); // 40-char lowercase hex
```

FR-006: Deduplicate Torrents by Infohash

- The union of all tracker URLs from all sources.
- The earliest `detectedAt` timestamp.

3.2 Integration and Transmission

FR-007: Send Magnet Links to qBitTorrent via WebUI API

Attribute	Value
Priority	P0 — Critical
Actor	API Client Module

The extension SHALL transmit magnet URIs to qBitTorrent's `/api/v2/tor` support for:

- `ur ls` parameter containing one or more magnet URIs (newline-separated).
- Optional `category`, `tags`, `savepath`, and `rename` parameters.
- Automatic cookie-based session management.

FR-008: Upload .torrent Files to qBitTorrent

Attribute	Value
Priority	P1 — High
Actor	API Client Module

The extension SHALL upload `.torrent` files via `multipart/form-data`

The extension SHALL expose a batch-send feature that:

- Enumerates all tabs within a selected Chrome tab group.
- Sends a `SCAN_REQUEST` to each tab's content script.
- Aggregates `SCAN_RESPONSE` results.
- Deduplicates across tabs.
- Transmits the unified list as a batch to the configured API.

FR-010: Enumerate Tab Groups and Extract URLs

Attribute	Value
Priority	P1 — High
Actor	Tab Manager Module

Using `chrome.tabGroups.query()` and `chrome.tabs.query()`, the ex

1. List all tab groups in the current window.
2. Display group titles and colors in the popup UI.
3. Allow the user to select one or more groups for batch operations.

3.3 Service Discovery and Authentication

FR-011: Auto-Discover Boba Services on Local Network

Attribute	Value
Priority	P2 — Medium

FR-012: Support Multiple Authentication Methods

Attribute	Value
Priority	P1 — High
Actor	Auth Module

The extension SHALL support the following authentication flows:

Method	Description
Cookie-based	Automatic <code>SID</code> cookie from qBitTorrent login
API Key	<code>X-API-Key</code> header for Boba server
Basic Auth	<code>Authorization: Basic <base64></code>
Custom Header	User-defined header name and value

FR-013: Encrypt Stored Credentials

Attribute	Value
Priority	P0 — Critical
Actor	Crypto Module

All authentication methods SHALL be supported by the extension.

FR-014: Offline Queue with Retry

Attribute	Value
Priority	P1 — High
Actor	Queue Manager Module

The extension SHALL maintain a persistent FIFO queue for torrents that fail

- Maximum queue size: 1,000 items (configurable).
- Retry schedule: exponential backoff starting at 5s, capped at 5 minutes.
- Maximum retry attempts: 5 (configurable).
- Queue items are persisted to `chrome.storage.local` and survive browser
- Failed items after max retries are moved to a "Dead Letter" sub-store for ma

FR-015: Real-time Download Progress via Badge

Attribute	Value
Priority	P2 — Medium
Actor	Badge Manager Module

The extension toolbar icon badge SHALL display:

- A numeric count of torrents queued for sending.
- Color coding: `green` (idle), `blue` (sending), `orange` (queued/retrying),
- Updates MUST occur within 200 ms of state changes.

Context	Label	Action
Link (magnet)	"Send Magnet to Boba"	Send
Link (.torrent)	"Download .torrent to Boba"	Down
Page	"Scan Page for Torrents"	Trigg
Tab Group	"Send Tab Group to Boba"	Batch

FR-017: Keyboard Shortcuts

Attribute	Value
Priority	P2 — Medium
Actor	Command Handler

The extension SHALL register the following shortcuts via `chrome.commands`

Command	Default Binding	Action
<code>send-current-page</code>	<code>Ctrl+Shift+B</code> (<code>Cmd+Shift+B</code> on macOS)	Scan page
<code>open-popup</code>	<code>Ctrl+Shift+L</code> (<code>Cmd+Shift+L</code> on macOS)	Open
<code>send-tab-group</code>	<code>Ctrl+Shift+G</code> (<code>Cmd+Shift+G</code> on macOS)	Send

The extension SHALL target the following browsers with a single codebase:

Browser	Minimum Version	Engine	Notes
Chrome	88+	Blink	Primary development
Firefox	109+	Gecko	Polyfill for <code>chrome</code>
Opera	74+	Blink	Chromium-based
Yandex Browser	21+	Blink	Chromium-based

Browser-specific code SHALL be isolated behind an `BrowserAdapter` abstraction.

FR-019: Internationalization Support

Attribute	Value
Priority	P2 — Medium
Actor	I18n Module

The extension SHALL support the following locales via `_locales/{lang}/`

Locale	Language
<code>en</code>	English (default)
<code>zh_CN</code>	Simplified Chinese

FR-020: Accessibility Compliance (WCAG 2.1 AA)

Attribute	Value
Priority	P1 — High
Actor	UI Layer

The extension UI SHALL meet WCAG 2.1 Level AA:

- Minimum contrast ratio 4.5:1 for normal text, 3:1 for large text.
- All interactive elements keyboard-navigable (Tab order, Enter/Space activation).
- ARIA labels on all icon-only buttons.
- Screen reader announcements for status changes (`aria-live` regions).
- Focus indicators visible on all focusable elements.

FR-021: Dark/Light Theme

Attribute	Value
Priority	P2 — Medium
Actor	UI Layer

The extension SHALL respect the system's preferred color scheme via `prefers-color-scheme` and provide a manual override toggle in Options. CSS custom properties SHALL be used for theme colors.

FR-022: Notifications for Download Events

Event	Title	Body
Send success	"Torrent Sent"	{name} added to download
Send failed	"Send Failed"	{name} — click to retry
Batch complete	"Batch Complete"	{count} torrents sent, {f
Queue retry	"Retrying Send"	Attempt {n} for {name}

Notifications SHALL be suppressible per-category in Options.

FR-023: Options Page Configuration

Attribute	Value
Priority	P1 — High
Actor	Options Page

The Options page SHALL provide configuration for:

- Server URL and connection mode (Boba proxy / qBitTorrent direct).
- Authentication credentials (with masked input and test-connection button).
- Default category, tags, and download path.
- Notification preferences.
- Queue settings (max size, retry count, backoff strategy).
- Keyboard shortcut display (non-editable; links to browser shortcuts page).
- Theme selection (system / light / dark).

The extension SHALL display a persistent connection status indicator:

- Polling interval: 30 seconds (configurable).
- Endpoint: `/api/v2/app/version` for qBitTorrent; `/health` for Boba.
- Status states: `connected`, `connecting`, `disconnected`, `error`.
- Visual indicator in popup header; tooltip shows last successful ping time.

FR-025: Rate Limiting for API Calls

Attribute	Value
Priority	P1 — High
Actor	API Client Module

The extension SHALL enforce client-side rate limiting:

- Default: 10 requests per second burst, 60 requests per minute sustained.
- Queue processing: maximum 1 concurrent API call with 500 ms inter-call delay.
- Respects HTTP 429 `Retry-After` header from server.
- All rate-limit state is held in-memory (non-persistent).

4. Non-Functional Requirements

4.1 Performance

ID	Requirement	Target	Measurement
NFR-001	Content script initialization	$\leq 10\text{ ms}$	performance
NFR-002	Magnet link detection (per link)	$\leq 5\text{ ms}$	Benchmark
NFR-003	Popup render time	$\leq 100\text{ ms}$	Lighthouse
NFR-004	Options page load time	$\leq 200\text{ ms}$	Lighthouse
NFR-005	API call round-trip (local network)	$\leq 500\text{ ms}$	95th percentile
NFR-006	Service worker cold start	$\leq 50\text{ ms}$	Chrome DevTools
NFR-007	Bundle size (compressed CRX)	$\leq 350\text{ KB}$	du -h

4.2 Security

ID	Requirement	Implementation
NFR-008	Credential encryption at rest	AES-256-GCM via Web Crypto
NFR-009	No cleartext credential logging	ESLint rule + code review
NFR-010	HTTPS-only API communication	URL scheme validation + CORS
NFR-011	Content Security Policy	<code>script-src 'self';</code>

4.3 Reliability

ID	Requirement	Target
NFR-013	Crash-free session rate	$\geq 99.9\%$
NFR-014	Offline queue durability	100% — all items
NFR-015	API compatibility coverage	qBitTorrent 4.4.x

4.4 Scalability

- Concurrent tab group batch size: unlimited (memory-bound).
- Offline queue: 1,000 items default, 10,000 maximum.
- Detection: pages with 10,000+ links must not cause UI jank (> 16 ms fr

4.5 Maintainability

- Code coverage: $\geq 80\%$ unit test coverage, $\geq 60\%$ E2E scenario coverage
- Documentation: all public APIs documented with JSDoc.
- TypeScript: strict mode enabled, zero `any` types in production code.
- Dependency count: ≤ 20 runtime dependencies.

5 System Architecture

5.2 Layer Architecture

Presentation Layer (Popup UI, Options Page, Context Menu)
Controller Layer (Message Router, Command Handler, Tab Manager)
Service Layer (Parser, API Client, Queue Manager, Health)
Security Layer (Auth Module, Crypto Module, Rate Limiter)
Storage Layer (chrome.storage.local, chrome.storage.session)

5.3 Interface Definitions

Content Script → Service Worker (chrome.runtime.sendMessage)

```
interface DetectionMessage {  
  type: 'TORRENT_DETECTED';  
  payload: {  
    torrents: TorrentInfo[];  
    pageUrl: string;  
    timestamp: number;  
  };  
}
```

```
interface ScanRequestMessage {
  type: 'SCAN_REQUEST';
  payload: {
    requestId: string;
    deepScan: boolean;
  };
}

interface ScanResponseMessage {
  type: 'SCAN_RESPONSE';
  payload: {
    requestId: string;
    torrents: TorrentInfo[];
    error?: string;
  };
}
```

6. Data Models

6.1 TorrentInfo

```
interface TorrentInfo {
    /** 40-character lowercase hex infohash */
    infohash: string;

    /** Human-readable torrent name */
    displayName: string;

    /** Magnet URI (reconstructed from parsed data) */
    magnetUri: string;

    /** Source type */
    source: 'magnet-link' | 'torrent-file';

    /** Announce URLs */
    trackers: string[];

    /** Total size in bytes, if known */
    totalSize?: number;

    /** Web seed URLs */
    webSeeds?: string[];

    /** URL of the page where detected */
    pageUrl: string;

    /** Timestamp of detection (Unix ms) */
    detectedAt: number;

    /** Raw torrent file blob (for .torrent uploads only) */
    fileBlob?: Blob;
}
```

6.2 ServerConfig

```
interface ServerConfig {
    /** Connection mode */
    mode: 'boba' | 'qbittorrent-direct';

    /** Base URL (e.g., https://boba.local:8443) */
    baseUrl: string;

    /** Authentication method */
    authMethod: 'cookie' | 'api-key' | 'basic' | 'custom-header';

    /** Credentials (encrypted at rest) */
    credentials: EncryptedCredentials;

    /** Default category for new torrents */
    defaultCategory?: string;

    /** Default tags for new torrents */
    defaultTags?: string[];

    /** Default save path */
    defaultSavePath?: string;

    /** Connection timeout in ms */
    timeout: number;

    /** Health check interval in ms */
    healthCheckInterval: number;
}
```

6.3 ExtensionConfig

```
interface ExtensionConfig {  
    /** Extension version that wrote this config */  
    version: string;  
  
    /** Server configuration */  
    server: ServerConfig;  
  
    /** UI preferences */  
    ui: {  
        theme: 'system' | 'light' | 'dark';  
        badgeEnabled: boolean;  
        notifications: {  
            sendSuccess: boolean;  
            sendFailed: boolean;  
            batchComplete: boolean;  
            queueRetry: boolean;  
        };  
    };  
  
    /** Queue behavior */  
    queue: {  
        maxSize: number;  
        maxRetries: number;  
        retryBaseDelayMs: number;  
        retryMaxDelayMs: number;  
    };  
  
    /** Detection preferences */  
    detection: {  
        scanDynamically: boolean;  
        maxTorrentFileSize: number;  
        enableTabGroupScan: boolean;  
    };  
};
```

6.4 QueueItem

```
interface QueueItem {  
    /** Unique queue entry ID */  
    id: string;  
  
    /** The torrent to send */  
    torrent: TorrentInfo;  
  
    /** Current status */  
    status: 'pending' | 'retrying' | 'failed' | 'dead-letter';  
  
    /** Number of send attempts made */  
    attemptCount: number;  
  
    /** Next scheduled retry time (Unix ms) */  
    nextRetryAt: number;  
  
    /** Last error message */  
    lastError?: string;  
  
    /** HTTP status code of last failure, if applicable */  
    lastHttpStatus?: number;  
  
    /** When the item was first queued */  
    queuedAt: number;  
}
```

7. API Specifications

Message Type	Direction	Payload
SCAN_REQUEST	SW → CS	{ requestId, deepScan }
SCAN_RESPONSE	CS → SW	{ requestId, torrents, error }
TORRENT_DETECTED	CS → SW	{ torrents, pageUrl, timestamp }
SEND_TORRENT	PU/SW → ACM	{ torrent, priority? }
SEND_BATCH	TM → ACM	{ torrents[] }
GET_STATUS	PU → SW	{ }
STATUS_RESPONSE	SW → PU	{ connected, queueSize, pendingCount }
GET_QUEUE	PU → QM	{ }
QUEUE_RESPONSE	QM → PU	{ items: QueueItem[] }
RETRY_ITEM	PU → QM	{ itemId }
REMOVE_ITEM	PU → QM	{ itemId }

```
POST /auth/token
```

```
Content-Type: application/json
```

```
{ "apiKey": "string" }
```

```
Response: { "token": "jwt-string", "expiresAt": 1700000000 }
```

Search API

```
GET /torrents/search?q={query}&category={category}&limit={limit}
```

```
Authorization: Bearer {token}
```

```
Response: { "results": TorrentInfo[], "total": number }
```

Download API

```
POST /torrents/download
```

```
Authorization: Bearer {token}
```

```
Content-Type: application/json
```

```
{ "infohash": "string", "category": "string?", "tags": ["string"] }
```

```
Response: { "jobId": "string", "status": "queued" }
```

SSE Streaming

```
GET /events/download-progress
```

```
Authorization: Bearer {token}
```

```
Accept: text/event-stream
```

```
Event format:
```

8. Security Architecture

8.1 Threat Model

Threat	Severity	Mitigation
Credential theft from disk	High	AES-256-GCM encryption; ke
Man-in-the-middle attack	High	HTTPS-only; certificate pinnin
XSS via malicious torrent site	Medium	CSP; content script isolation; n
Privilege escalation	Medium	Minimal permission manifest;
Extension fingerprinting	Low	No unique identifiers exposed

8.2 Encryption Flow

[Diagram: System Architecture Overview]

8.3 Content Security Policy

```
{
  "content_security_policy": {
```

9.1 Error Categories

Code	Category	Retry Strategy
E_NETWORK	Network failure (DNS, TCP)	Exponential backoff
E_TIMEOUT	Request timeout	Immediate retry × 2, then backoff
E_AUTH	Authentication failure	No retry; manual intervention required
E_RATE_LIMIT	HTTP 429	Honor Retry-After , then backoff
E_VALIDATION	Invalid torrent data	No retry
E_SERVER	5xx server error	Backoff with jitter
E_STORAGE	Local storage full	No retry; alert user

9.2 Error Object Schema

```
interface ExtensionError {
  code: 'E_NETWORK' | 'E_TIMEOUT' | 'E_AUTH' | 'E_RATE_LIMIT'
  message: string;
  timestamp: number;
  context?: Record<string, unknown>;
}
```

- BLOB references for `.torrent` files are revoked immediately after up
`URL.revokeObjectURL()`.
- Service worker uses lazy initialization for all modules; unused modules

10.2 DOM Observation Strategy

```
// Optimized MutationObserver
const observer = new MutationObserver((mutations) => {
  const hasLinkMutations = mutations.some(m =>
    Array.from(m.addedNodes).some(n =>
      n instanceof HTMLElement &&
      (n.tagName === 'A' || n.querySelector('a'))
    )
  );
  if (hasLinkMutations) {
    debouncedScan();
  }
});

observer.observe(document.body, { childList: true, subtree: t
```

- Scanning is debounced at 250 ms to batch rapid DOM mutations.
- `requestIdleCallback` is used for non-urgent parsing on pages with

10.3 Bundle Optimization

- Tree-shaking enabled via WXT/Vite.
- Code-splitting: Options page and Popup are separate chunks.
- Shared vendor chunk for crypto and parsing utilities.

11. Technology Stack

Layer	Technology	Version	Purpose
Language	TypeScript	≥ 5.0	Type-safe de
Framework	WXT	≥ 0.17	Extension b
Bundler	Vite	≥ 5.0	Module bun
Testing (Unit)	Jest	≥ 29.0	Unit and int
Testing (E2E)	Playwright	≥ 1.40	Browser aut
Linting	ESLint	≥ 8.0	Code quality
Formatting	Prettier	≥ 3.0	Code format
Crypto	Web Crypto API	Native	AES-256-G
Storage	Extension Storage API	MV3	chrome.stor
UI	Vanilla TS + CSS	Native	No framewo

11.1 Runtime Dependencies

bencode-js	^3.0.0	Bencode encoder/decoder for .torre
lz-string	^1.5.0	Compression for queue persistence

12. Deployment Architecture

12.1 Build Outputs

```
dist/
├─ chrome/                # Chrome/Opera/Yandex build (MV3)
│   ├─ manifest.json
│   ├─ background.js
│   ├─ content-scripts/
│   ├─ popup/
│   ├─ options/
│   └─ _locales/
├─ firefox/               # Firefox build (MV3 with polyfills)
│   └─ ...
└─ edge/                  # Edge build (identical to Chrome)
    └─ ...
```

12.2 CI/CD Pipeline



12.3 Distribution Channels

Channel	Format	Update Mech
---------	--------	-------------

13. Assumptions and Dependencies

13.1 Assumptions

- 1. The target qBitTorrent instance has the WebUI enabled and accessible v
- 2. Users have administrative access to configure Boba server or qBitTorren
- 3. The local network allows HTTP(S) communication between the browse
- 4. Users will grant the `activeTab` permission when prompted for site-sp
- 5. The browser supports Manifest V3 (all targeted versions do).

13.2 External Dependencies

Dependency	Minimum Version
Node.js	18 LTS
qBitTorrent (WebUI)	4.4.0
Boba Server	1.0.0
Chrome / Chromium	88
Firefox	109

14. Risks and Mitigations

Risk	Probability	Impact	Mitigation
Chrome policy changes restrict MV3 capabilities	Medium	High	Maintain MV2 fallback Chromium feature r
qBitTorrent WebUI API breaking changes	Medium	Medium	Version detection; a
CORS blocking torrent file downloads	High	Medium	Proxy downloads th notification
User forgets encryption password	Medium	Medium	Optional password l reconfigure
Extension store policy rejection	Low	High	Pre-submission revi practices
Third-party dependency vulnerability	Medium	High	Automated Depend scanning
Large pages causing performance issues	Medium	Medium	<code>requestIdleCa</code> link sets

15. Appendices

Appendix A: Glossary

Term	Definition
Infohash	A 20-byte (40-character hex) SHA-1 hash uniquely identifying a
Magnet URI	A URI scheme (<code>magnet:</code>) containing metadata for peer-to-peer
Bencode	The encoding format used by BitTorrent for <code>.torrent</code> files; s integers, and byte strings.
Boba	The orchestration server that proxies and manages torrent downl
qBitTorrent	An open-source BitTorrent client with a built-in WebUI API.
Content Script	A script injected by the extension into web pages to read and mo
Service Worker	The event-driven background script in MV3 that handles API cal script messaging.
MV3	Manifest V3, the third version of the WebExtension manifest for
WXT	A modern build tool for browser extensions that wraps Vite.
mDNS	Multicast DNS, a protocol for name resolution on local networks
CSP	Content Security Policy, a browser mechanism to control resourc
AES-256-GCM	Advanced Encryption Standard with 256-bit key in Galois/Coun encryption.

- 4. [qBitTorrent WebUI API Documentation](#)
- 5. [Chrome Extension Manifest V3](#)
- 6. [Mozilla WebExtensions API](#)
- 7. [WXT Framework Documentation](#)
- 8. [WCAG 2.1 Specification](#)
- 9. [Web Crypto API Specification](#)
- 10. [Server-Sent Events \(MDN\)](#)

Appendix C: Document History

Version	Date	Author	Changes
0.1.0	2026-05-15	Platform Team	Initial draft
0.2.0	2026-05-28	Platform Team	Added security
0.3.0	2026-06-02	Architecture Board	Review feedback
1.0.0	2026-06-06	Platform Team	Final release

End of Technical Specification